

Daftar Isi

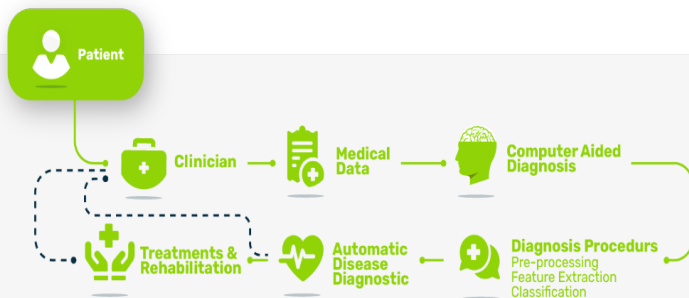
Kata Pengantar.....	iv
Daftar Isi.....	v
Bab I Pengenalan <i>Artificial Intelligence</i> dan <i>Machine Learning</i>	1
1.1. Apa itu <i>Artificial Intelligence</i> (AI)?.....	2
1.1.1. <i>Intelligent System</i> (IS), <i>Intelligent Agent</i> (IA), dan AI	4
1.1.2. Kategori IA.....	7
1.1.3. Manfaat Perkembangan AI	8
1.1.4. AI Akan Menutup Lapangan Kerja Manusia?!... 10	
1.2. Beberapa Jenis Model AI.....	12
1.2.1. <i>Expert System</i> (ES)	12
1.2.2. <i>Fuzzy Inference System</i> (FIS)	14
1.2.3. Model AI yang belajar dari data	17
1.3. <i>Machine Learning</i> (ML)	18
1.4. <i>Deep Learning</i> (DL)	19
Rangkuman	22
Saatnya Berlatih!.....	24
Bab II <i>Machine Learning</i> : Model AI yang Seperti Murid.....	31
2.1. Bagaimana Model <i>Machine Learning</i> Berlatih?	31
2.1.1. <i>Supervised Learning</i>	32
2.1.2. <i>Unsupervised Learning</i>	34
2.1.3. <i>Reinforcement Learning</i> (RL)	36
2.2. <i>Parameter</i> dan <i>Hyperparameter</i>	38
2.3. Apa Itu <i>Loss</i> ?	40

2.4. <i>Gradient Descent</i>	42
2.5. Metrik Evaluasi Untuk Model ML	43
Rangkuman	44
Saatnya Berlatih!	

Bab I

Pengenalan *Artificial Intelligence* dan *Machine Learning*

Dengan revolusi industri 4.0, sistem cerdas atau sistem canggih dapat ditemukan di sekitar kita. Sebagai contoh, saat kita mengambil foto dengan kamera pada *smartphone*, kita dapat secara otomatis memfokuskan kamera pada wajah manusia menggunakan fitur *face detection* pada saat kita melakukan swafoto. Bahkan di dunia medis pun, banyak sistem *Computer-Aided Diagnosis* (CAD) yang bisa membantu dokter dalam melakukan pemeriksaan pada para pasiennya. Tentunya semua teknologi ini tidak lepas dari sistem cerdas, di mana teknologi *Artificial Intelligence* (AI) sudah berperan. Namun untuk melakukan tugas-tugas di atas, akan sangat memakan waktu jika kita memprogram rangkaian aturan satu persatu untuk setiap jenis penyakit demi membuat sistem cerdas tersebut. Untuk mengatasi masalah ini, teknologi *Machine Learning* dapat menjadi solusinya. Tetapi, apakah sebenarnya yang dimaksud dengan *Machine Learning*? Apa kaitan *Machine Learning* dengan AI? Untuk menjawab pertanyaan-pertanyaan ini, kita harus mengetahui terlebih dahulu definisi AI dan *Machine Learning*.



Gambar 1.1 Ilustrasi *Computer-Aided Diagnosis* (CAD)

1.1. Apa itu *Artificial Intelligence* (AI)?

Di era ini, AI sudah menjadi istilah yang sangat tidak asing bagi kita. Terkadang, kita mengkategorikan berbagai teknologi yang modern dan canggih sebagai AI, baik itu berupa *hardware* seperti mesin otomatis pada pabrik ataupun *software* seperti teknologi pengenalan wajah untuk verifikasi pada *smartphone*. Namun apa definisi AI yang sebenarnya? Apakah AI adalah *hardware* seperti robot manusia (*humanoid*), *software*, algoritma, atau suatu bidang ilmu yang sedang berkembang?



Gambar 1.2 Ilustrasi *Humanoid*

Sejak tahun 1950-an, berbagai peneliti telah mencetuskan berbagai definisi yang bervariasi tentang AI, seperti yang tertulis pada **Tabel 1.1**. McCarthy, Ric, dan Hassabis mendefinisikan AI sebagai suatu bidang ilmu atau mata pelajaran. Namun seiring berkembangnya zaman, definisi seperti yang dibeirkan Kaushik pun muncul, yang menganggap bahwa AI berupa mesin. Namun, definisi menurut Ribeiro adalah yang paling menarik karena pendapatnya berbeda jauh dengan yang lainnya, di mana ia beranggapan bahwa AI bukan hanya sekedar mesin, melainkan “teknologi” yang memungkinkan mesin bertindak seperti manusia. Dengan kata lain menurutnya, AI bisa dianggap berupa suatu perangkat lunak (*software*) yang diprogram sedemikian rupa untuk menirukan cara berpikir manusia. Namun apakah definisi seperti ini sudah tepat?

Tabel 1.1 Berbagai Definisi AI Menurut Berbagai Tokoh

Pencetus	Definisi
John McCarthy (1955)	Ilmu dan teknik untuk membuat mesin yang dapat berperilaku cerdas
Elaine Rich (1983)	Studi mengenai bagaimana cara membuat komputer dapat melakukan pekerjaan yang pada saat ini lebih berhasil dilakukan oleh manusia di banding mesin
David L. Poole & Alan K. Mackworth (2010)	Agen komputasi yang bertindak secara cerdas
Demis Hassabis (2015)	Ilmu pengetahuan untuk membuat mesin menjadi pintar
Avinash Kaushik (2017)	Mesin yang cerdas
Jair Ribeiro (2021)	Satu jenis teknologi yang menirukan cara berpikir manusia dan memungkinkan mesin untuk bertindak dan berfungsi sendiri seperti kecerdasan manusia, seperti kemampuan untuk memahami, mempelajari, bernalar, dan bertindak



Gambar 1.3 John McCarthy, Bapak dari AI

1.1.1. *Intelligent System (IS), Intelligent Agent (IA), dan AI*

Beberapa orang kebingungan dalam membedakan AI dengan *Intelligent System* ataupun *Intelligent Agent*. Bayangkan kita membangun sebuah robot. Sebuah robot yang dibangun untuk membantu pekerjaan manusia di pabrik dapat disebut sebagai sebuah sistem yang cerdas (*Intelligent System*). Namun, ada juga yang menyebutnya sebagai agen cerdas (*Intelligent Agent*), bahkan ada juga yang menyebutnya sebagai AI. Lantas, apa perbedaan antara ketiga istilah ini?

Berikut ini beberapa definisi mengenai istilah-istilah tersebut diatas:

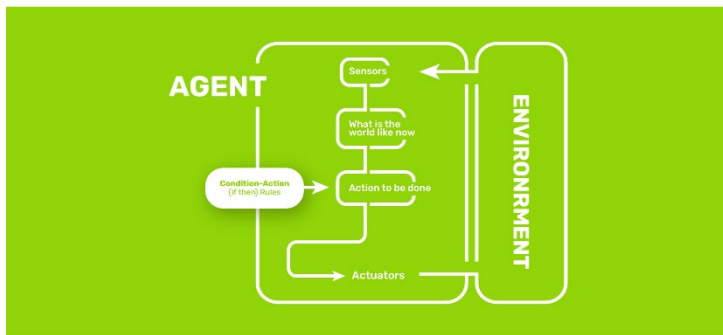
Martin Molina (2020) mendefinisikan *Intelligent System* sebagai berikut:

“Sebuah sistem yang: (a) beroperasi pada suatu lingkungan bersama agen, (b) memiliki kemampuan kognitif seperti mengerti, mengontrol tindakan, bernalar, maupun berbahasa, (c) bertindak secara rasional, serta (d) memiliki kemampuan untuk beradaptasi dengan belajar.”

Sedangkan istilah *Intelligent Agent* sendiri dapat didefinisikan sebagai berikut oleh Russel dan Norvig (2003):

“Entitas otonom yang mampu mengerti lingkungan sekitarnya, mengambil tindakan untuk mencapai tujuannya, serta mampu meningkatkan performanya dengan belajar atau menggunakan pengetahuannya.”

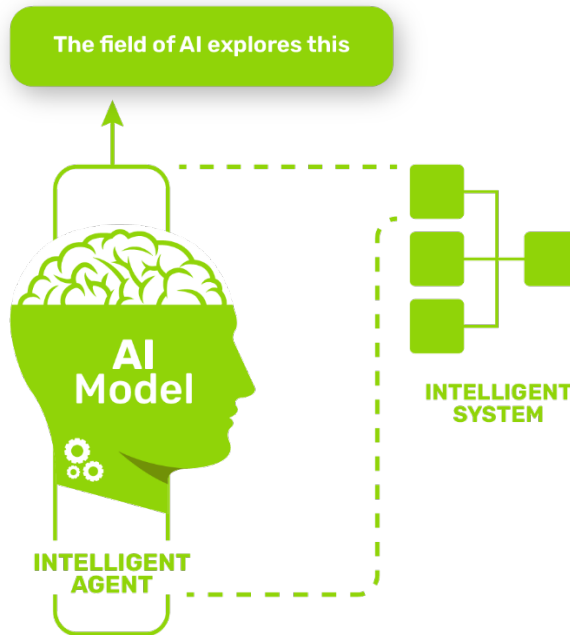
Berdasarkan definisi tersebut diatas serta ilustrasi pada **Gambar 1.4**, sistem kontrol cerdas sederhana seperti *thermostat* pun dapat dikategorikan sebagai IA. Dalam kasus *thermostat*, sensor digunakan untuk membaca kondisi lingkungan sekitarnya, dan data hasil bacaan tersebut kemudian diproses oleh IA untuk menentukan apakah aktuator seperti *heater* ataupun kipas perlu digunakan. Kunci utama dari sebuah IA adalah kemampuan untuk beroperasi secara otomatis setelah ia menerima data dari sensor.



Gambar 1.4 Diagram Sederhana Untuk *Reflex Intelligent Agent*

Berdasarkan definisi dari McCarthy, Rich, dan Hassabis, pada awalnya AI diartikan sebagai bidang ilmu yang mempelajari IA. Namun, pergeseran makna seiring berkembangnya zaman membuat istilah AI juga digunakan untuk menyebut IA dan IS.

Sebagaimana dijelaskan oleh Poole dan Mackworth (2010), istilah AI pun sudah tidak jarang digunakan untuk mewakili IA sebagai agen komputasi yang cerdas. Padahal, ketiga istilah ini memiliki arti yang berbeda. IA dan IS sendiri memiliki definisi yang sangat



Gambar 1.5 Ilustrasi Hubungan AI, IA, dan IS

mirip, di mana keduanya merujuk pada entitas yang dapat memahami kondisi lingkungan, memproses data tersebut, dan mengaktifkan aktuator. Namun perbedaan besar dalam definisi kedua istilah ini adalah, IS merujuk pada sistem cerdas yang beroperasi pada suatu lingkungan bersama agen termasuk manusia, sedangkan IA adalah entitas otonom yang memiliki kemampuan memahami lingkungan dan mengambil tindakan. Dengan kata lain, IS membutuhkan bantuan agen seperti manusia dalam menjalankan fungsinya, sedangkan IA dapat beroperasi sendiri untuk setiap input yang ia deteksi secara otomatis

menggunakan sensor. Maka itu, IA bisa digambarkan sebagai sebuah *software* ataupun *hardware* cerdas di dalam IS, di mana sebuah IS bisa tersusun atas satu atau lebih IA. Di dalam program IA terdapat model kecerdasan buatan (model AI), yang dapat didefinisikan sebagai *software* yang memproses data *input* dengan menggunakan pengetahuan yang ada di dalamnya untuk menghasilkan *output*. AI sendiri dapat didefinisikan dengan merujuk kepada definisi dari McCarthy, yaitu bidang ilmu yang mempelajari mengenai agen cerdas (IA).

1.1.2. Kategori IA

Model AI yang ada di dalam IA memungkinkannya memiliki kecerdasan untuk menjalankan berbagai fungsi. Secara umum, terdapat 4 kategori IA, yaitu sebagai berikut.

- *Acting humanly*

Konsep IA yang bisa bertindak seperti manusia (*acting humanly*) pertama kali diajukan dengan adanya *Turing test*. *Turing test* sendiri adalah suatu uji coba di mana seorang manusia yang disebut *interrogator* akan memberikan pertanyaan tertulis kepada dua *player*, di mana salah satu dari kedua *player* adalah sebuah komputer dan satunya lagi manusia. IA tersebut dikatakan berhasil jika *interrogator* tidak mampu membedakan responsnya dengan respons manusia. Dalam hal ini, IA diekspektasikan memiliki kemampuan *Natural Language Processing* (untuk berkomunikasi dalam bahasa Inggris), *knowledge representation* (untuk menyimpan informasi yang diketahui dan diterimanya), *automated reasoning* (menggunakan informasi yang dimiliki untuk menjawab dan menarik kesimpulan), serta *machine learning* (untuk beradaptasi dengan situasi baru, mengenali, serta memprediksi pola).

- *Thinking humanly*

Selain *acting humanly*, ada juga IA yang diharapkan bisa berpikir seperti manusia (*thinking humanly*). Dengan menggunakan pendekatan *cognitive*

modeling, sebuah program bisa dibuat menyerupai cara manusia berpikir dalam memecahkan masalah. Dengan kata lain, program yang dapat memecahkan suatu masalah dengan algoritma atau langkah-langkah yang menyerupai cara manusia dapat dikatakan sebagai IA.

- *Thinking rationally*

Berpikir rasional berarti menghasilkan kesimpulan yang tepat dari premis yang tepat sesuai asas silogisme. IA diharapkan memiliki kemampuan yang demikian, namun IA dengan prinsip ini sangat sulit untuk diterapkan karena terdapat perbedaan besar antara masalah yang didesain waktu pengujian IA dengan masalah yang ada di dunia di mana IA akan digunakan.

- *Acting rationally*

Dengan pendekatan *rational agent*, IA dapat didefinisikan sebagai agen yang merasakan lingkungannya dan mengambil tindakan rasional untuk mencapai tujuannya. Dalam bidang AI secara logika, tindakan rasional di sini adalah penentuan *output* yang tepat berdasarkan *input* yang diterima. Namun jika terdapat situasi di mana ada *uncertainty* atau situasi di mana tidak ada tindakan yang menghasilkan *best outcome*, maka IA harus mampu mengambil tindakan yang menghasilkan *best expected outcome*.

1.1.3. Manfaat Perkembangan AI

Dengan berkembangnya IS dan banyaknya IA di sekitar kita, banyak sekali dampak positif yang dapat kita rasakan. Salah satunya adalah automasi. Mungkin kita sudah sering melihat adanya IA seperti *chatbot* atau *virtual assistant* pada berbagai IS, termasuk *smartphone* kita yang dapat membantu kita menjalankan fungsi pada *smartphone* hanya dengan perintah suara. Contoh lainnya adalah pemantauan penggunaan masker untuk pendeteksian pelanggaran protokol kesehatan secara otomatis. Perkembangan AI telah memacu banyaknya implementasi IA untuk memperoleh manfaat automasi, akurasi

(meminimalkan *human error*), serta optimalisasi pelaksanaan tugas manusia. Contoh manfaat perkembangan AI dalam berbagai bidang kehidupan dapat dilihat pada contoh-contoh berikut ini.

- *Cyber Security*:

Seperti yang kita ketahui, terdapat model AI di dalam IA dan beberapa jenis model AI memiliki kemampuan untuk mempelajari pola dari data. Jenis model AI yang disebut *Machine Learning* (ML) ini banyak sekali digunakan di dunia *cyber security*, khususnya untuk mempelajari dan mendeteksi pola *cyber-attack*, serta mencegahnya. IS dengan kemampuan ini disebut *Intrusion Detection System* (IDS). IDS memungkinkan pemantauan keamanan *cyber* dapat dilakukan secara otomatis tanpa terlalu mengandalkan penilaian manusia dan mengurangi potensi terjadinya *human error* atau kesalahan penilaian oleh manusia. Implementasi teknologi AI lainnya dalam bidang ini adalah deteksi informasi sensitive dan deteksi *botnet*, yaitu jaringan computer yang sering digunakan untuk menyebabkan DDoS (*Distributed Denial of Service*).

- *Medis*:

Dengan berkembangnya ML dan *Deep Learning* (DL), berbagai model AI telah dikembangkan untuk membantu tenaga medis untuk mendiagnosis penyakit, sesuatu yang dikenal dengan sebutan CAD. CAD dapat menganalisis gambar medis seperti gambar CT atau X-Ray. Namun di bidang biomedis, berbagai model AI juga sudah dikembangkan untuk melakukan studi di bidang genomik untuk mengembangkan *personalized medicine* untuk penyakit seperti kanker.

- *Pemasaran (marketing)*:

Dalam bidang *marketing*, teknologi AI juga sudah banyak digunakan untuk pembuatan konten, *lead scoring*, *advertisement targeting*, serta *dynamic pricing*. Hal-hal ini juga didukung oleh adanya teknologi ML dan DL. Dengan menggunakan

unsupervised learning, ML bisa digunakan untuk membuat *association model*, dan menganalisis pola produk atau jasa yang banyak dibeli untuk membuat prediksi penjualan di masa depan.

- Transportasi:

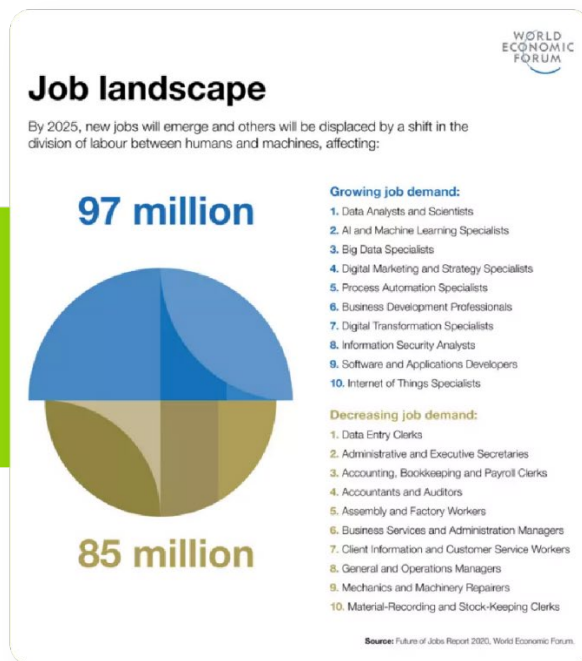
Sebagai salah satu inovasi populer di zaman ini, *self-driving car* adalah salah satu contoh implementasi IA di dalam teknologi yang kita gunakan setiap hari, yaitu kendaraan. Di dalam sistem *self-driving car*, terdapat berbagai jenis sensor yang dapat membaca data lingkungan sekitarnya seperti kamera termografis, radar, sonar, GPS, dan sebagainya yang terhubung dengan berbagai IA. IA yang ada di dalam sistem ini meliputi IA untuk melakukan navigasi dan perencanaan rute perjalanan, *sensing* dan mengemudikan kendaraan, pemantau kesiagaan pengemudi, dan sebagainya. Dalam bidang ini, manfaat automasi dan optimalisasi telah terwujud. Para peneliti juga berharap adanya sistem *self-driving car* dapat mengurangi angka kecelakaan yang umumnya terjadi karena kurangnya kewaspadaan pengemudi, yang termasuk dalam kategori *human error*.

1.1.4. AI Akan Menutup Lapangan Kerja Manusia?!

Dalam sebuah studi pada tahun 2016 yang dipublikasikan oleh *World Economic Forum* (WEF), disebutkan bahwa industri 4.0, era di mana teknologi AI akan banyak diimplementasikan, dapat menutup lebih dari lima juta pekerjaan dalam kurun waktu 5 tahun. Hal ini disebabkan oleh manfaat automasi yang diberikan oleh teknologi AI dalam berbagai bidang pekerjaan, sehingga banyak pekerjaan manusia dapat dikerjakan oleh IA. Pernahkah kalian berpikir demikian? Lantas, apakah perkembangan teknologi AI bisa dikatakan sebagai sesuatu yang positif atau negatif?

Sebenarnya, pernyataan bahwa AI dapat menutup berbagai lapangan pekerjaan manusia memang tidak dapat dibantah. Dengan adanya manfaat automasi, akurasi, dan optimalisasi dari

AI, manusia tidak perlu membuang tenaga terlalu banyak untuk melakukan berbagai pekerjaan secara manual. Sebagai contoh, kita dapat melihat di beberapa perusahaan bahwa kini tugas *customer service* dapat dibantu dengan adanya *chatbot*. Namun di balik hal ini, tentu ada juga lapangan pekerjaan baru, seperti AI *engineer* untuk mengembangkan teknologi *chatbot*. Dengan kata lain, AI dapat menutup berbagai lapangan kerja, tetapi ia juga dapat membuka berbagai lapangan kerja baru. Tentunya, lapangan kerja baru ini tidak terbatas pada teknisi AI saja.



Gambar 1.6 Prediksi Peningkatan dan Penurunan Permintaan Tenaga Kerja untuk Beberapa Profesi Menurut WEF

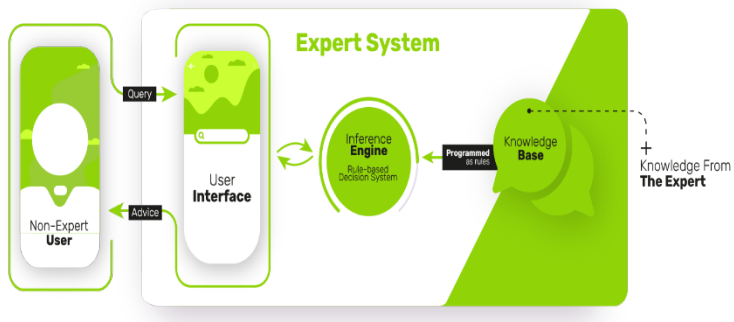
WEF memperkirakan hingga tahun 2025, akan terjadi perpindahan karir yang besar dengan pembagian kerja antara manusia dan mesin. Bahkan diperkirakan bisa tercipta 97 juta

pekerjaan baru untuk manusia, di mana beberapa di antaranya sudah muncul di era ini, seperti *data scientist*, *AI specialist*, *digital transformation specialist*, dan sebagainya. Sebaliknya, permintaan akan pekerjaan *data entry* dan administrasi diperkirakan akan mengalami penurunan. Dapat disimpulkan bahwa perkembangan teknologi AI memang dapat menutup beberapa lapangan kerja, tapi bukan berarti akan terjadi era di mana manusia tidak lagi bekerja. Kehadiran AI akan membawa lapangan kerja baru bagi manusia, sehingga manusia-pun perlu beradaptasi untuk hidup bersama dan bekerjasama dengan AI (*co-exist and co-operate*).

1.2. Beberapa Jenis Model AI

Dari subbab sebelumnya, kita sudah mengenal apa yang disebut dengan AI, IA, serta IS. Kita pun tahu bahwa di dalam IA, terdapat model AI atau model kecerdasan yang memungkinkannya menjalankan fungsi. Seperti apakah model AI tersebut? Apakah bentuknya seperti algoritma? Subbab ini akan membahas mengenai berbagai jenis model AI yang cenderung umum digunakan dalam berbagai IS.

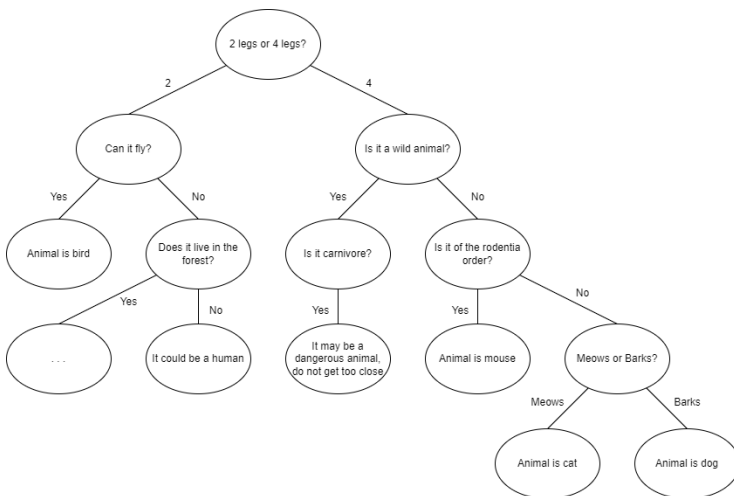
1.2.1. Expert System (ES)



Gambar 1.7 Ilustrasi *Expert System*

Expert System (sistem pakar) dikenal juga sebagai *knowledge-based system* atau *rule-based system* dan pertama kali diperkenalkan sekitar tahun 1970-an. Model AI yang merupakan

salah satu jenis model tertua ini memiliki kemampuan untuk meniru kemampuan *decision-making* dari seorang pakar. Hal ini dilakukan dengan menyusun rangkaian *rules* ke dalam sebuah program, yang nantinya akan disebut *decision system*. Rangkaian *rules* ini dapat diibaratkan seperti sebuah diagram pohon. Sebagai contoh, sebuah ES untuk klasifikasi binatang, seperti pada **Gambar 1.8**, dapat disusun oleh berbagai *rules* untuk setiap jenis binatang yang dapat diklasifikasi oleh sistem, di mana ciri-ciri masing-masing binatang akan dijadikan *rules* tersebut. Setiap *rules* akan diprogram sebagai rangkaian algoritma seleksi atau “*if-then*”, sehingga ketika model ES digunakan, jawaban dari masing-masing *rule* akan digunakan untuk menghasilkan *output*. *Rules* inilah yang menjadi *knowledge* atau pengetahuan oleh model ES. Dalam prakteknya, rangkaian *rules* ini ditentukan oleh pakar dari bidang di mana ES akan digunakan, sehingga model AI ini disebut sistem pakar.



Gambar 1.8 Contoh *Expert System* Untuk Klasifikasi Binatang

Karena tersusun atas rangkaian aturan “*if-then*”, ES tergolong jenis AI yang dapat bekerja dengan cepat dan sederhana. Pengguna juga dapat membaca dengan jelas

penjelasan untuk setiap *output* yang diperoleh dengan memperhatikan *rules* yang dipenuhi oleh data *input*. Implementasinya juga tergolong mudah sebagai model AI, dan konsep *multiple expertise* juga dapat diterapkan pada IA di mana ES diterapkan, yaitu konsep di mana lebih dari satu model ES dapat digunakan pada satu IA untuk menyelesaikan suatu masalah.

Namun, salah satu kelemahan besar dari ES adalah pembuatannya, lebih tepatnya proses pengumpulan *rules* atau *knowledge* (disebut sebagai *knowledge acquisition*). Pembuat ES perlu berkonsultasi dengan pakar untuk mendata semua *rules* yang diperlukan dan memasukannya ke dalam program satu per satu. Hal ini tentunya sangat memakan waktu dan tenaga. Selain itu, ada kemungkinan adanya *rules* penting yang tidak terprogram karena tidak diketahui pakar yang menjadi konsultan. Ditambah lagi dengan ketidakmampuan ES untuk memproses data mentah, ES menjadi model AI yang tidak banyak digunakan untuk masalah-masalah kompleks. Namun jika tugas yang akan dikerjakan ES memiliki *rules* yang sederhana, ES tetap menjadi pilihan yang baik.

1.2.2. Fuzzy Inference System (FIS)

Dalam bidang AI, konsep *fuzzy logic* telah melahirkan model yang disebut FIS. *Fuzzy logic* sendiri adalah salah satu model AI yang berbentuk *many-valued logic* di mana nilai kebenaran dari setiap variabel bisa berupa bilangan real apa saja di antara 0 sampai 1, yang disebut dengan *fuzzy value*. FIS adalah model *rule-based system* yang menggunakan konsep ini, tidak seperti ES biasa yang cenderung menggunakan *crisp value* (nilai yang hanya berupa ya atau tidak untuk setiap variabel). Dengan kata lain, FIS adalah model ES yang didesain untuk menghadapi data dengan nilai yang “tidak pasti” atau *fuzzy*. Sebuah nilai *input* dirubah menjadi bentuk *fuzzy value* dengan menggunakan formula matematis yang disebut *fuzzy membership function* dengan contoh sebagai berikut.

Misalkan kita memiliki data *input* berupa suhu ruangan dalam derajat Celsius, dan data ini akan digunakan oleh

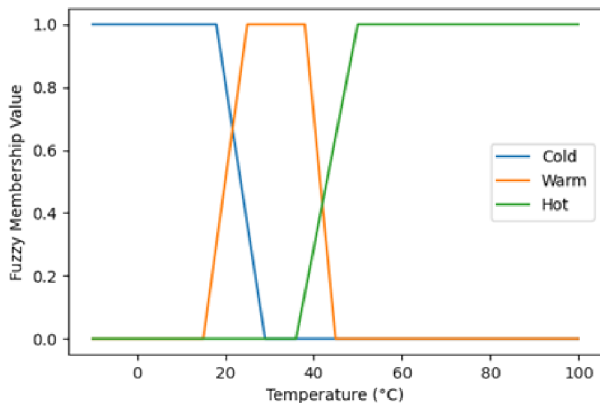
sistem *thermostat* untuk menentukan penggunaan AC. Dengan menggunakan FIS, misalnya kita menentukan *fuzzy membership function* berikut ini.

$$\mu_{Cold}(x) = \begin{cases} 1 & , x \leq 18 \\ \frac{29-x}{11} & , 18 < x < 29 \\ 0 & , x \geq 29 \end{cases}$$

$$\mu_{Warm}(x) = \begin{cases} 0 & , x \leq 15 \\ \frac{x-15}{10} & , 15 < x < 25 \\ 1 & , 25 < x < 38 \\ \frac{45-x}{7} & , 38 < x < 45 \\ 0 & , x \geq 45 \end{cases}$$

$$\mu_{Hot}(x) = \begin{cases} 0 & , x \leq 36 \\ \frac{x-36}{14} & , 36 < x < 50 \\ 1 & , x \geq 50 \end{cases}$$

di mana setiap nilai suhu x akan memiliki *fuzzy membership value* μ_{Cold} untuk kelas “Cold”, μ_{Warm} untuk kelas “Warm”, dan μ_{Hot} untuk kelas “Hot”. *Fuzzy membership value* untuk setiap kelas dapat divisualisasikan pada **Gambar 1.9**.



Gambar 1.9 Contoh Visualisasi *Fuzzy Membership Value* untuk Variabel Temperatur

Salah satu jenis FIS yang paling populer dikenalkan oleh Ebrahim H. Mamdani pada tahun 1974. Model yang disebut “*Mamdani’s FIS*” ini bekerja sebagai berikut:

- 1) *Fuzzification* untuk mengubah nilai *crisp input* menjadi *fuzzy membership value*
- 2) Pengaktifan *rules* pada *rule-based system*,
- 3) *Defuzzification* untuk mengubah *fuzzy membership value* dari *output* menjadi *crisp value*.

Rule-based system yang digunakan di sini sangat mirip dengan ES yang tersusun atas aturan “*if-then*”. Nilai *output* yang diperoleh dari rangkaian aturan ini berupa *fuzzy membership value*, yang kemudian akan dikembalikan menjadi *crisp value* dengan menggunakan metode *center-of-weight*. Selain model FIS Mamdani, ada juga model FIS yang disebut model TSK (Takagi-Sugeno-Kang), di mana proses *defuzzification* dilakukan pada *rule-based system*.

Secara umum, model FIS termasuk model AI yang masih umum digunakan, termasuk dalam pemeriksaan kualitas udara serta prediksi gempa. Model ini cenderung membutuhkan lebih

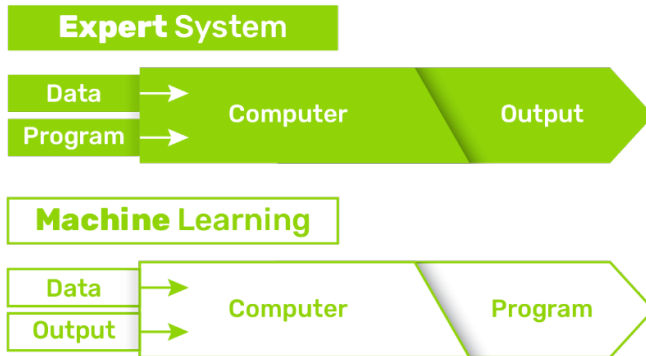
sedikit *rules* jika dibandingkan dengan ES, serta memiliki kemampuan untuk menangani faktor ketidakpastian atau mengambil nilai “tengah-tengah” dari data *crisp*. Namun mirip dengan ES, model AI ini didesain untuk diprogram secara manual, sehingga dibutuhkan pengumpulan *rules* yang jelas terlebih dahulu, serta pemrograman setiap *rules* yang digunakan secara manual.

1.2.3. Model AI yang belajar dari data

Dengan berkembangnya zaman, jenis data yang dapat dianalisa juga menjadi semakin variatif. Pada awal perkembangan ES dan FIS, manusia berpikir bagaimana caranya membuat mesin meniru manusia dapat melakukan pekerjaan manusia. Namun, peneliti pun mulai menganalisis bagaimana caranya membuat mesin meniru cara manusia belajar. Sebagai anak-anak, kita tidak selalu menerima “*input*” dan “*proses*” untuk menghitung “*output*”. Ada saatnya, kita menerima “*input*” dan “*output*”, tapi kita perlu memahami prosesnya. Misalnya saat kita melihat foto seorang selebriti, kita mungkin hanya diberitahu foto dan namanya, dan kita pun mulai mengenali sendiri karakteristik wajah selebriti tersebut. Konsep ini pun memicu lahirnya model AI yang dapat mempelajari “*proses*” ini, atau yang kita sebut “*pola*”, “*keterkaitan*”, ataupun “*korelasi*” antar data.

Dalam dunia nyata, terdapat berbagai jenis hubungan antar variabel, misalnya korelasi linear, *causal-effect* atau sebab-akibat, dan sebagainya. Dengan adanya paradigma yang disebut *Machine Learning* (ML), para peneliti pun mengembangkan model AI yang mampu menganalisis hubungan antar data ini. Kehadiran ML memungkinkan peneliti AI untuk tidak lagi memprogram satu persatu *rules* yang diperlukan untuk suatu pekerjaan secara manual. Dengan kata lain, model ML mampu mempelajari *rules* dari data secara otomatis jika kita memberikannya data *input* dan *output*. Hal inilah yang membedakan ML dengan ES maupun FIS, di mana dalam membuat ES dan FIS kita membutuhkan data dan program *rules* untuk mendapatkan *output*, sedangkan dalam membuat model ML kita membutuhkan data dan *output* untuk menyusun model ML dengan *rules* yang ia pelajari sendiri.

Ilustrasi perbedaan ini dapat dilihat pada **Gambar 1.10**



Gambar 1.10 Ilustrasi Perbedaan ES dan ML

1.3. *Machine Learning* (ML)

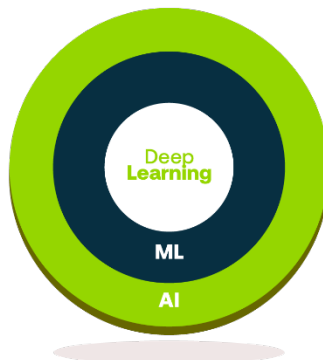
Sebelumnya, kita sudah mengetahui bahwa sebuah IA dapat dikatakan cerdas jika ia dapat meningkatkan performanya dengan belajar. Lebih tepatnya, model AI di dalam IA adalah bagian yang belajar dan menyimpan *knowledge* di dalamnya. ML adalah bidang ilmu yang mempelajari cara membuat model AI dapat belajar. Umumnya, model AI ini disebut sebagai model ML, yaitu model algoritma komputasi yang mampu menyimpan serta mengubah *knowledge* yang ia miliki melalui proses pembelajaran dari data. Model ML adalah salah satu jenis model AI yang paling banyak digunakan di era 4.0, baik untuk data berupa tabel, teks, suara, dan sebagainya.

Model ML menyimpan pengetahuan mereka dalam bentuk parameter, yang dapat direpresentasikan sebagai *rules* atau angka bilangan bulat (*integer*) maupun decimal (*float*). Namun, model ML yang menggunakan *rules* sebagai representasi *knowledge* memiliki kemampuan untuk memperbaharui *rules* tersebut berdasarkan data yang digunakan untuk melatihnya. Proses melatih model ML menggunakan data ini disebut dengan *training*. Model ML yang dilatih ini umumnya akan diujikan pada data lain yang belum pernah

ia lihat sebelumnya, yang disebut sebagai *independent test set*. Penjelasan lebih detail mengenai ML serta proses *training* dari model ML akan kita bahas pada bab II.

Seperti yang sudah dibahas pada subbab sebelumnya, kelebihan utama ML adalah kemampuannya dalam mempelajari dan memperbaharui *knowledge* yang ia miliki secara otomatis. Hal ini memungkinkan peneliti AI untuk tidak memprogram *rules* secara manual, tapi mereka dapat membiarkan model ML mempelajari sendiri pola dan keterkaitan antar variabel secara otomatis, yang mana merupakan manfaat yang efektif dan efisien dalam berbagai bidang kehidupan. Namun, salah satu kelemahan besar model ML adalah ketergantungannya pada data. Karena model ML bekerja dengan menganalisis *training data* yang diberikan kepadanya, kualitas data yang buruk (misalnya banyaknya *noise*, *outlier*, dan sebagainya) dapat merusak kinerja model ML secara signifikan. Pada bab III, kita akan mengupas aspek-aspek data yang baik yang dibutuhkan oleh model ML.

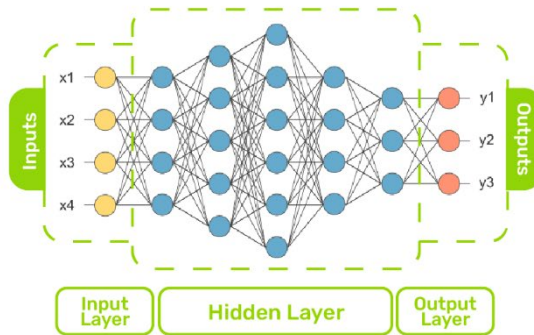
1.4. *Deep Learning* (DL)



Gambar 1.11 Ilustrasi Hubungan AI, ML, dan DL

Salah satu cabang ilmu dari ML yang sangat populer pada era 4.0 adalah DL. DL sendiri dapat didefinisikan sebagai sub-bidang ML yang berfokus pada model AI yang berbasis pada *Artificial Neural*

Network (ANN) dengan menggunakan konsep *representation learning*. *Representation learning* sendiri adalah rangkaian metode komputasi yang memungkinkan mesin untuk menganalisa data mentah (*raw data*) dan mengubahnya menjadi bentuk representasi data lain yang dapat dipahami mesin. Hal ini umumnya dilakukan dengan menggunakan ANN sebagaimana digambarkan pada **Gambar 1.12**, di mana *hidden layer* digunakan untuk menyusun *intermediate feature representation*, yaitu bentuk fitur abstrak yang mewakili pola dari data *input*.



Gambar 1.12 Ilustrasi *Representation Learning* pada ANN

ANN sendiri adalah model tiruan dari jaringan saraf manusia yang tersusun atas kumpulan neuron yang berlapis-lapis. Jaringan saraf manusia tersusun atas sistem saraf pusat / *Central Nervous System* (CNS) dan sistem saraf tepi / *Peripheral Nervous System* (PNS). CNS pada manusia adalah otak dan sumsum tulang belakang, sedangkan PNS tersusun atas neuron sensorik (saraf pengirim rangsangan ke CNS) dan neuron motorik (saraf pengirim sinyal dari otak ke otot sebagai respons rangsangan). ANN pun tersusun atas *input layer* yang bertindak seperti neuron sensorik, *hidden layer* yang

merepresentasikan CNS, dan *output layer* yang bertindak seperti neuron motorik dan menghasilkan *output*. Di dalam setiap *layer*, terdapat neuron yang bertindak seperti sel pada otak.

Dalam menyimpan representasi *knowledge*, ANN menggunakan parameter yang disebut *weight* dan *bias*. Berdasarkan ilustrasi pada **Gambar 1.12**, *weight* sendiri adalah garis-garis yang menghubungkan masing-masing neuron, sedangkan *bias* adalah nilai di luar neuron yang akan mempengaruhi *output* dari setiap neuron. Seperti model ML pada umumnya, nilai *weight* dan *bias* ini berupa nilai *integer* atau *float* untuk merepresentasikan *knowledge*. Dengan kata lain, pemrosesan *input* di dalam ANN tersusun atas proses matematis dengan menggunakan *weight* dan *bias*.

Secara umum, terdapat 3 jenis model ANN yang umum digunakan:

- *Convolutional Neural Network* (CNN), jenis ANN yang menggunakan operasi matriks yang disebut *convolution* untuk mengekstrak fitur spasial. Jenis ini banyak digunakan untuk menganalisis data visual seperti gambar atau video.
- *Recurrent Neural Network* (RNN), jenis ANN yang menggunakan konsep pemrosesan sekuensial untuk menganalisis data dalam bentuk *sequence*. RNN banyak digunakan untuk mengekstrak fitur temporal atau urutan data, misalnya analisis *sequence* protein, data *time-series*, suara, teks, dan sebagainya.
- *Deep Neural Network* (DNN), yaitu salah satu jenis ANN yang paling dasar. Umumnya, ANN dengan kurang dari 2 *hidden layer* tidak disebut sebagai model DL. Hal inilah yang menjadi landasan istilah “*Deep Learning*”, yaitu ML dengan menggunakan ANN yang *deep* dengan banyak *layer*. Jenis ANN ini umumnya digunakan untuk menganalisis data dalam bentuk variabel yang sudah diekstrak, seperti data tabel.

Selain ketiga jenis ANN di atas, sekarang sudah muncul berbagai variasi model DL lainnya, misalnya *Graph Neural Network*, *Time Delay Neural Network*, dan sebagainya. Dapat dikatakan bahwa DL sudah menjadi bidang ilmu yang sudah sangat berkembang dan diminati di dunia ML. Hal ini didukung juga dengan kelebihan utama model DL, yaitu kemampuannya untuk mengekstrak fitur secara otomatis. CNN dapat memproses dan menganalisis data gambar

secara otomatis, RNN dapat menganalisis data berbentuk *sequence* secara langsung, dan model-model ini mampu menentukan sendiri fitur yang memiliki pengaruh signifikan terhadap *output*. Namun, kemampuan DL yang luar biasa ini juga diimbangi oleh kekurangannya. Proses matematika di balik model DL adalah operasi matriks, yang membutuhkan kemampuan pemrosesan paralel (*parallel computing*) yang besar. Karena itu, penelitian menggunakan DL cenderung membutuhkan *hardware* dengan kapabilitas demikian seperti *graphics processing unit* (GPU), serta *library* pendukungnya seperti CUDA dari NVIDIA.

Saatnya Berlatih!

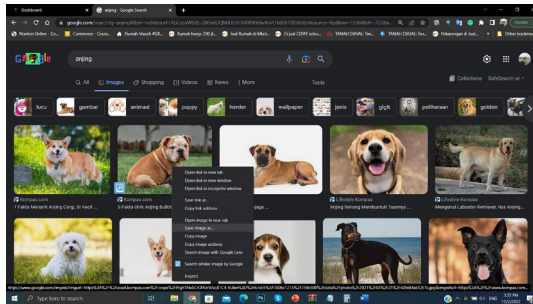
Google Teachable Machine: Membuat Model *Deep Learning* (DL) Untuk Pemula

Google Teachable Machine adalah sebuah situs web dari Google yang dapat mempercepat dan memudahkan pembuatan model DL, tanpa perlu membuat *source code* secara manual. Teachable machine bekerja dengan cara mengumpulkan data ke dalam kelas-kelas atau kategori kemudian melatih sebuah model DL berdasarkan kelas yang telah dibentuk.

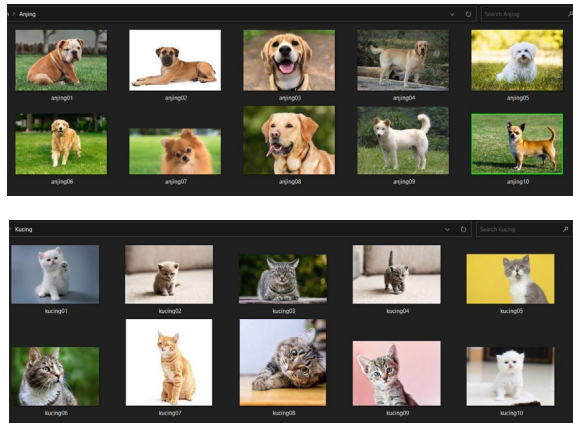
Pada latihan kali ini, kita akan mencoba melatih suatu model DL untuk membedakan gambar anjing dan kucing. Ikuti langkah-langkah berikut ini!

Langkah 1. Kumpulkan dan simpan (minimal 10) gambar/ foto kucing dan anjing.

Anda dapat mencari koleksi foto/ gambar tersebut melalui situs pencarian images.google.com dengan memasukkan kata kunci anjing atau kucing. Pilih satu gambar kemudian klik kanan mouse, pilih perintah *save as image*, dan simpan pada folder yang sudah Anda tentukan. Lakukan berulang sampai minimal 10 gambar kucing dan 10 gambar anjing. *Screenshot* proses pencarian gambar tampak pada Gambar 1.13, sedangkan contoh data gambar/ foto anjing dan kucing tampak pada Gambar 1.14.

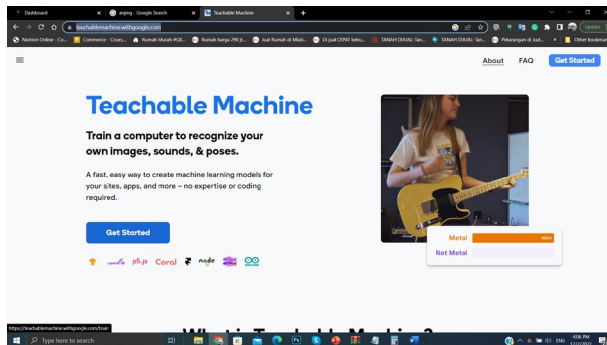


Gambar 1.13 *Screenshot* Proses Pencarian Gambar pada Langkah 1

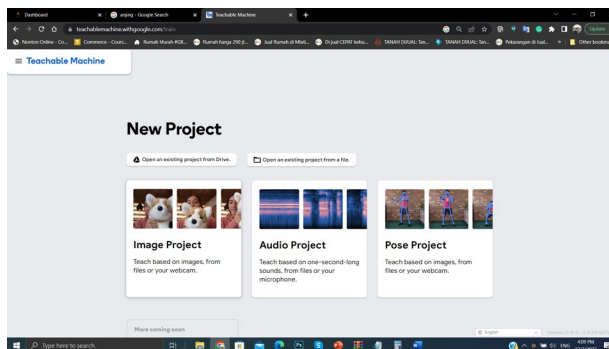


Gambar 1.14 Contoh Gambar Kucing dan Anjing

Langkah 2. Kita akan mulai menggunakan *Google Teachable Machine* (GTM) dengan membuka situs web teachablemachine.withgoogle.com (tampilan layar akan tampak seperti Gambar 1.15.) dan tekan tombol “Get Started” untuk membuka pilihan proyek/ task yang bisa dilakukan dalam pekerjaan (tampil layar akan tampak seperti Gambar 1.16).

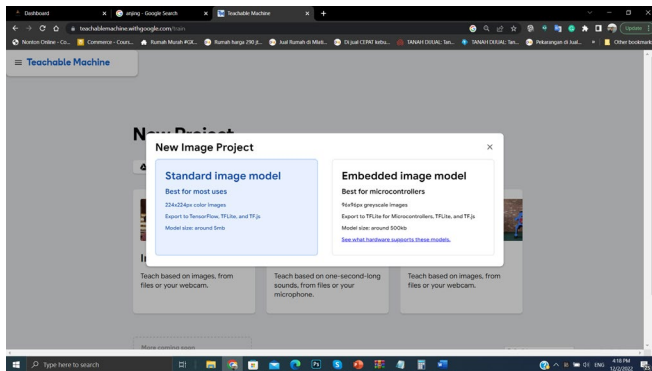


Gambar 1.15 *Screenshot* Situs GTM

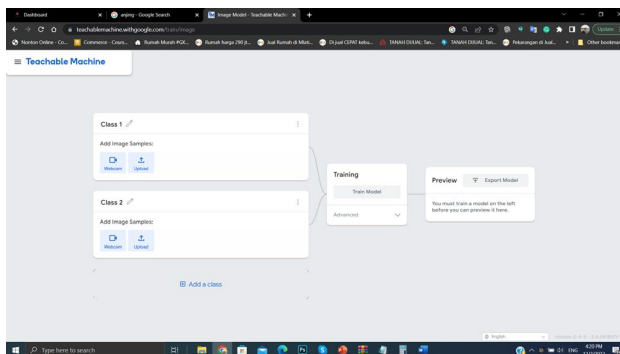


Gambar 1.16 *Screenshot* Pilihan Proyek dalam GTM

Langkah 3. Pilih menu “*Image Project*” karena data yang akan digunakan model DL kita adalah data yang berupa gambar/ foto. Setelah menu dipilih akan tampil pilihan menu model gambar (tampil layar seperti Gambar 1.17), kemudian pilih tombol “*Standard image model*” untuk memulai dashboard proyek (tampil layar seperti Gambar 1.18.)

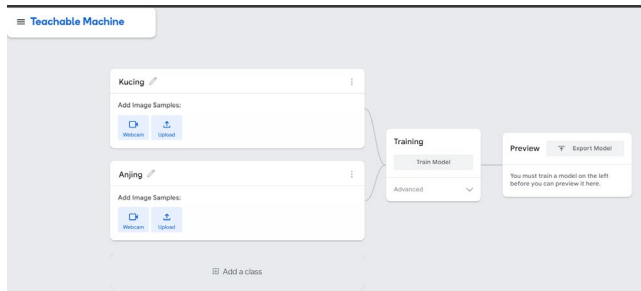


Gambar 1.17 *Screenshot* Pilihan Model Gambar



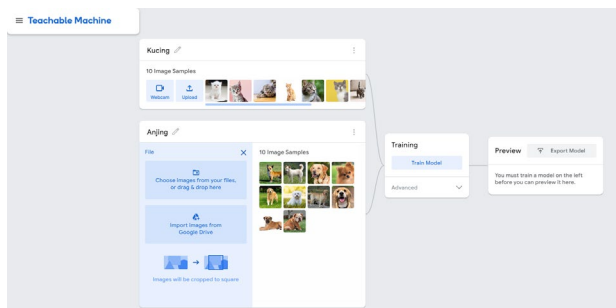
Gambar 1.18 *Dashboard* Proyek Gambar

Langkah 4. Tentukan nama kelas pada setiap kategori dengan cara klik icon  dan ubah “Class 1” menjadi “Kucing” dan “Class 2” menjadi “Anjing”. Hasil tampilan layar akan tampak seperti Gambar 1.19.



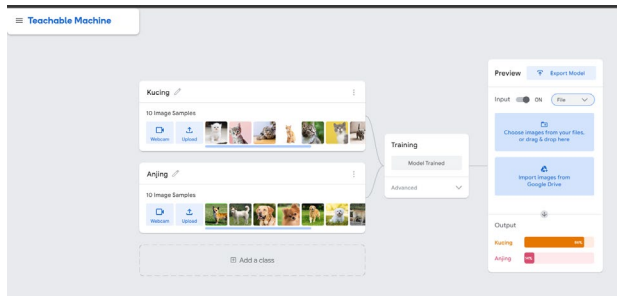
Gambar 1.19 Screenshot Nama Kelas yang Telah Diubah

Langkah 5. Upload data ke portal GTM dengan cara klik ikon  (upload) pada kelas Kucing, pilih “Choose image from your file”, arahkan ke folder tempat Anda menyimpan semua gambar kucing pada Langkah 1. Pilih semua gambar kucing kemudian klik tombol “Open”. Lakukan juga untuk kelas Anjing.



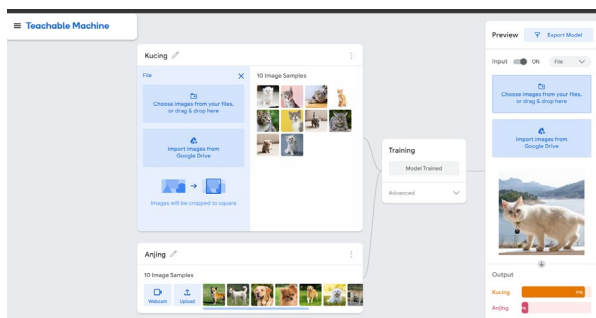
Gambar 1.20 Screenshot Data yang Telah Ter-upload

Langkah 6. Lakukan *training* dengan menekan tombol “*Train Model*” dan tunggu sampai proses selesai.



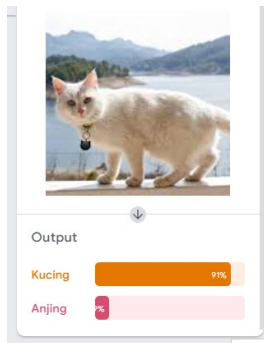
Gambar 1.21 *Screenshot* Proses Training yang Telah Selesai

Langkah 7. Lakukan ujicoba model dengan cara cari dan upload 1 gambar anjing atau kucing (selain gambar yang telah dipakai untuk training) pada bagian tab *Preview*, dengan menekan tombol “*Choose images from your files*”.



Gambar 1.22 *Screenshot* Proses Ujicoba Model

Langkah 7. Lakukan pengamatan hasil ujicoba model dengan memperhatikan kolom *output* pada tab *preview*, seperti tampak pada Gambar 1.22.



Gambar 1.23 Tampilan Output Ujicoba

Bab II

Machine Learning: Model AI yang Seperti Murid

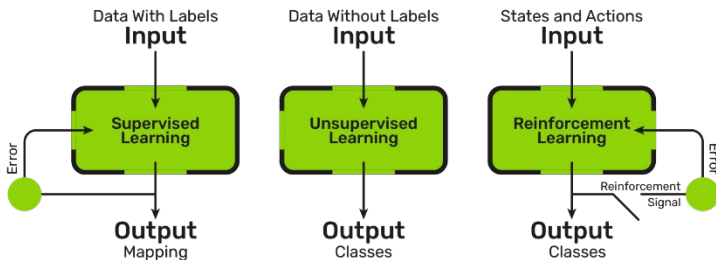
Pada bab sebelumnya, kita telah berkenalan dengan konsep dasar dari ML, serta perbedaan dari AI, ML, dan DL. ML dan DL berperilaku seperti manusia, tepatnya seperti murid yang sedang mempelajari sesuatu, yaitu menganalisis pola dan *rules* dari data yang mereka terima. Hal tersebut dilakukan karena ML dan DL ditujukan untuk menirukan perilaku cerdas manusia, baik untuk membuat keputusan maupun melakukan suatu prediksi. Namun mungkin kita kembali bertanya, bagaimana model ML bisa memiliki kemampuan untuk mempelajari pola data dan menyimpan representasi *knowledge* di dalamnya? Bagaimana model ML tersebut belajar? Bagaimana cara model ML mempelajari pola dari data tersebut? Bagaimana kita menilai performa model ML yang telah kita kembangkan? Jawaban akan pertanyaan-pertanyaan tersebut kami ulas dalam sub-bab berikut ini.

2.1. Bagaimana Model *Machine Learning* Berlatih?

Melatih (*training*) sebuah model ML bisa diartikan sebagai proses penentuan *knowledge* yang sesuai untuk dimiliki model ML. Untuk kasus ANN, hal ini berarti menentukan nilai *weight* dan *bias* yang tepat untuk model yang sedang dilatih agar ia dapat menghasilkan *output* yang tepat untuk kasus yang sedang dipelajarinya. Proses *training* ini dilakukan hingga model ML mampu melakukan generalisasi, yaitu memahami pengetahuan umum dalam tugas yang harus model ML tersebut lakukan.

Satu hal yang harus kita ketahui adalah tidak semua model ML dilatih dengan cara yang sama. Ada tiga cara yang dapat digunakan untuk melatih model ML, yaitu *supervised learning*, *unsupervised learning*, dan *reinforcement learning*. Masing-masing metode ini

menggunakan cara yang berbeda untuk memberikan *feedback* atau umpan balik kepada model ML selama proses *training*. Ketiga metode ini dapat diimplementasikan sesuai dengan tugas yang nantinya akan dikerjakan oleh model ML tersebut.

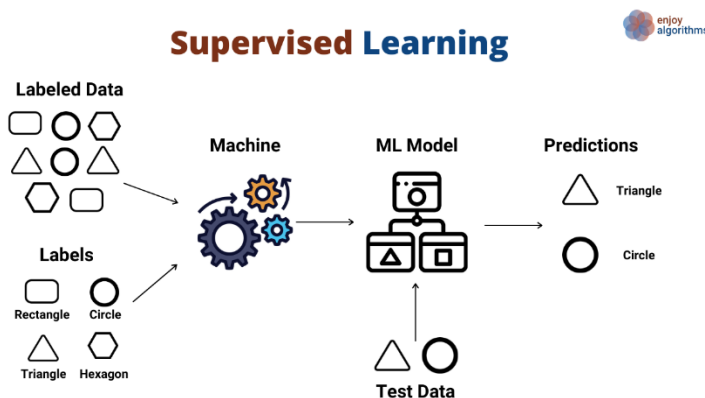


Gambar 2.1 Ilustrasi Perbedaan *Supervised Learning*, *Unsupervised Learning*, dan *Reinforcement Learning*

2.1.1. *Supervised Learning*

Metode *supervised learning* bisa juga disebut “*learning by example*”, yaitu metode melatih model ML di mana sebuah model diberikan *input* data beserta dengan *output* berupa label dari *input* tersebut. Bayangkan kita sedang mengajarkan seorang anak untuk mengerjakan soal matematika, tapi kita juga memberikannya kunci jawaban untuk dicocokkan nantinya. Sang anak akan mencoba menjawab soal-soal yang ia terima, lalu ia akan membandingkan jawabannya dengan kunci. Pada setiap jawaban yang salah, ia akan memperbaharui *knowledge* yang ia miliki sebelumnya. *Supervised learning* bekerja seperti siklus yang dialami anak tersebut, di mana model ML belajar dari “contoh soal” dengan jawaban, atau yang biasa disebut dengan data *input* dan *output* diberikan kepada model untuk dipelajari. Biasanya, data *output* ini disebut sebagai *label*. Dengan kata lain, *supervised learning* dapat didefinisikan sebagai metode *training* model ML di mana model akan menganalisis data *input* untuk menghasilkan *prediction*, dan *prediction* ini akan dibandingkan dengan *label*

yang diberikan untuk melatih model menghasilkan *label* yang tepat. Perbedaan hasil ini disebut sebagai *loss* yang nantinya akan digunakan untuk memperbaharui *knowledge* pada model ML. *Loss* inilah yang disebut juga sebagai *feedback*. Pembahasan yang lebih mendalam mengenai *loss* dan penggunaannya untuk memperbaharui *knowledge* akan kita bahas pada subbab 2.3

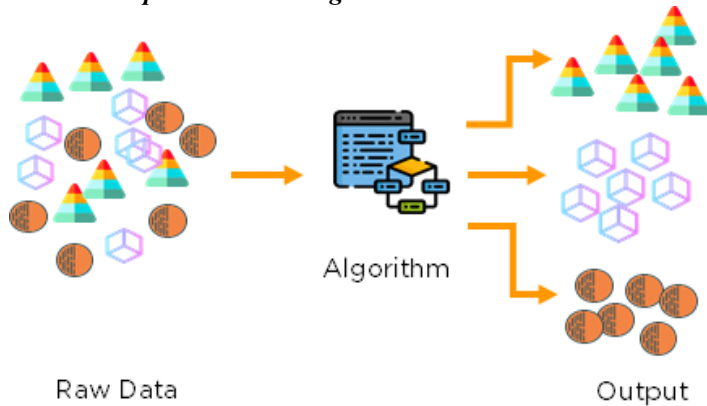


Gambar 2.2 Ilustrasi Proses *Supervised Learning* (Raj, n.d.)

Metode *learning* ini sangat umum digunakan dalam berbagai bidang ML, khususnya *regression* dan *classification*. Dalam bidang ML, *regression* adalah *task* untuk membuat prediksi yang berupa nilai kontinu. Beberapa contoh kasus *regression* ini adalah prediksi jumlah pembeli suatu produk atau barang, prediksi rating film, perkiraan tingkat keparahan suatu penyakit yang diderita seseorang, dan sebagainya. Di lain sisi, *classification* adalah *task* yang berupa pengelompokkan data ke dalam kelompok dengan *label*. Dalam *task* ini, model ML akan menganalisis fitur kunci yang membedakan data dari masing-masing kelompok. Contoh kasus untuk *task* ini adalah klasifikasi binatang seperti yang dicontohkan pada bab I, klasifikasi jenis penyakit, deteksi *email* spam, serta *sentiment analysis*, dan sebagainya. Beberapa jenis model ML yang umum digunakan dengan metode *supervised learning* meliputi *Linear Regression*,

Logistic Regression, Decision Tree (DT), Support Vector Machine (SVM), ANN, Extreme Gradient Boosting (XGBoost), dan sebagainya.

2.1.2. *Unsupervised Learning*



Gambar 2.3 Ilustrasi *Unsupervised Learning* (Sharma, 2019)

Berbeda dengan *supervised learning*, metode *unsupervised learning* tidak membutuhkan *label* untuk melatih model ML. Model yang dilatih dengan *unsupervised learning* diarahkan untuk menganalisis pola tersembunyi dari data *input* yang mereka terima, kemudian mengelompokkan data-data yang mirip. Karena tugasnya adalah mengelompokkan data yang mirip, metode ini tidak membutuhkan *label* pada data. Metode ini disebut juga “*learning by reasoning*”, di mana model ML berlatih untuk bernalar dan menentukan fitur yang mirip ataupun berbeda dari masing-masing data. Teknik *unsupervised learning* secara umum dapat digunakan untuk empat jenis kasus:

- *Clustering*
Clustering sendiri adalah *task* yang paling mendekati definisi *unsupervised learning*, di mana tugas model *clustering* adalah mengelompokkan data-data yang mirip ke dalam sebuah *cluster* dan menjauhkannya dari

cluster data lain yang tidak mirip. *Clustering* merupakan salah satu *task* yang banyak diteliti di bidang ML. Contoh kasus untuk *clustering* adalah *market segmentation* dan *customer churn prediction*.

- *Association*

Association adalah *task* untuk menganalisis hubungan/asosiasi antara objek data dengan menganalisis hubungan antar variabel di dalamnya. Contoh kasus untuk *association* adalah pembuatan *recommendation engine* pada aplikasi pemutar video atau lagu.

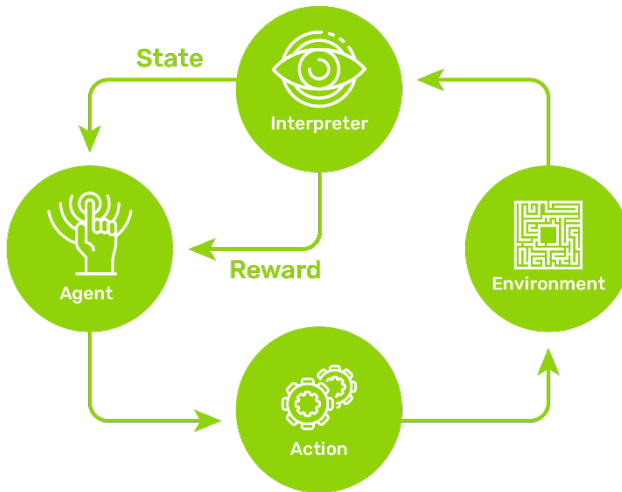
- *Anomaly Detection*

Dengan menggunakan kemampuan seperti *clustering*, *unsupervised learning* memungkinkan model ML untuk mengelompokkan data-data dengan keanehan, atau yang biasa disebut dengan *anomaly*. Contoh implementasi model ML untuk *task* ini meliputi *fault diagnosis* pada sistem, deteksi transaksi *fraud*, serta deteksi *network security intrusion*.

- *Dimensionality Reduction*

Model ML yang dilatih menggunakan *unsupervised learning* juga dapat digunakan untuk menyederhanakan data dengan dimensi tinggi menjadi representasi data dengan dimensi rendah. Hal ini dikarenakan model ML tersebut dapat mengidentifikasi fitur-fitur kunci dari masing-masing kelompok data, apa yang membuat suatu data mirip dengan data lainnya, tapi berbeda dari kelompok data lain. Dengan mengambil fitur-fitur kunci tersebut saja, kita bisa mendapatkan representasi dimensi rendah dari setiap data.

2.1.3. Reinforcement Learning (RL)



Gambar 2.4 Ilustrasi Reinforcement Learning

Pada bab I, kita sudah membahas mengenai IA. Secara teknis, RL adalah cara model ML berlatih yang paling mirip dengan konsep IA. Model ML yang dilatih dengan RL umumnya disebut juga *agent*, karena model ini diharapkan mampu memahami kondisi *environment* dan belajar untuk mengambil *action* terbaik. *Agent* dilatih untuk mengenali kondisi *environment* dan belajar untuk mengambil *action* yang dapat memberikannya *reward*. Pengambilan *action* ini didasarkan pada konsep yang disebut *exploration* dan *exploitation*. *Exploration* adalah *action* yang dilakukan untuk mengetahui kondisi atau fitur baru yang belum diketahui *agent* pada *environment* dengan mengambil *action* yang tidak optimal. Sedangkan *exploitation* adalah *action* yang dilakukan untuk mencapai *reward* maksimal berdasarkan apa yang sudah diketahui oleh *agent* tentang *environment*. *Exploration* dilakukan agar *agent* dapat mencoba *action* baru yang mungkin bisa menghasilkan *reward* yang lebih baik. Dengan kata lain, keseluruhan proses RL dapat dijelaskan sebagai proses

pelatihan *agent* dalam suatu *environment* dengan melatihnya mengambil *action* yang tepat untuk mencapai *reward* yang optimal.

Reward dalam RL bisa dikatakan sebagai nilai yang menunjukkan seberapa baik akibat dari *action* yang dilakukan *agent*. Jika dibandingkan dengan *supervised learning* dan *unsupervised learning*, *reward* ini memiliki peran yang mirip dengan *loss*, di mana nilai *reward* ini yang akan digunakan untuk memperbaharui *knowledge* di dalam *agent*. Sistem *reward* ini bisa disebut sebagai *reward and penalty*. *Reward* adalah nilai yang mengindikasikan *agent* mengambil tindakan yang tepat dan berdampak positif pada *state*, sedangkan *penalty* adalah nilai yang menandakan *agent* mengambil tindakan yang salah dan berdampak negatif pada *state*. Metode *reinforcement learning* ini sangat umum digunakan dalam pembuatan AI untuk *game*.

Berdasarkan ketiga metode pelatihan model ML yang kita bahas di atas, kita dapat melihat bahwa masing-masing metode diperuntukkan untuk kasus yang berbeda-beda. **Tabel 2.1** merangkum perbedaan dari masing-masing metode. Selain ketiga metode ini, sebenarnya masih ada metode lain yang dapat digunakan untuk melatih model ML, seperti *semi-supervised learning* serta *evolutionary algorithm*. Namun karena masih terdapat berbagai hal yang belum kita bahas maka 2 metode tersebut tidak kami ulas pada buku ini. Hal-hal yang perlu kita pelajari selanjutnya adalah bagaimana nilai *loss* dihitung? Bagaimana model ML memperbaharui *knowledge* yang ia miliki berdasarkan *loss*? Nah, sebelum kita membahas hal-hal ini, kita perlu mengetahui juga dua istilah penting dalam ML, yaitu *parameter* dan *hyperparameter*, yang akan dibahas pada subbab 2.2 berikut ini.

Tabel 2.1 Rangkuman Perbedaan Cara Model ML Berlatih

Aspek	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Data	Membutuhkan data dengan label	Membutuhkan data tapi tidak membutuhkan label	Tidak membutuhkan data
Cara Belajar	Belajar dari contoh, sesuai dengan label data	Belajar dengan bernalar untuk menentukan fitur yang penting	Melakukan exploration dan exploitation untuk membangun knowledge tentang action yang tepat
Tujuan Model	Mengerjakan 1 task spesifik yang dilatih saja	Menemukan insight dari kumpulan data tanpa perlu memberi label	Mempelajari action yang tepat dalam environment yang tidak pasti

2.2. *Parameter dan Hyperparameter*

Dalam melatih ML, ada dua istilah penting yang harus kita ketahui, yaitu *parameter* dan *hyperparameter*. Kedua aspek inilah yang ditentukan nilai optimalnya dalam keseluruhan proses *training* model ML. Tapi apa sebenarnya perbedaan keduanya? Dalam konteks ML, *parameter* adalah variabel yang nilainya ditentukan dan dipelajari dari data yang digunakan. Dengan kata lain, *parameter* bisa dikatakan merupakan variabel yang merepresentasikan *knowledge* pada model, seperti *weight* dan *bias* pada ANN ataupun *rules* pada *decision tree*. Sebaliknya, *hyperparameter* adalah variabel konfigurasi yang berada di luar model ML serta tidak dipengaruhi oleh data yang digunakan. Jika diibaratkan seperti manusia, *parameter* menandakan pengetahuan yang kita miliki, sedangkan *hyperparameter* adalah pengaturan seperti waktu belajar dan jumlah buku yang kita baca. Untuk memahami perbedaannya secara lebih dalam kita dapat melihat beberapa contoh dari *hyperparameter*.

Sebagai variabel konfigurasi *training*, *hyperparameter* akan sangat mempengaruhi performa model ML. Nilai dari *hyperparameter* sendiri perlu ditetapkan secara manual oleh pembuat

model ML. Nilai-nilai ini nantinya akan digunakan untuk mengatur berapa lama model akan dilatih, seberapa besar perubahan nilai *parameter* akan dilakukan, formula perhitungan perubahan nilai *parameter* dan sebagainya. Beberapa contoh dari *hyperparameter* yang digunakan untuk melatih model ML seperti ANN dapat dilihat pada **Tabel 2.2**. Kita dapat melihat bahwa *hyperparameter* adalah variabel konfigurasi. Rangkuman dari perbedaan *hyperparameter* dengan *parameter* dapat kita lihat pada **Tabel 2.3**.

Tabel 2.2 Beberapa Contoh *Hyperparameter* yang Digunakan dalam Melatih Model ANN

Nama	Definisi
Learning Rate	Hyperparameter yang menentukan seberapa besar perubahan nilai weight dan bias akan dilakukan dalam satu step proses training
Batch Size	Jumlah data dalam 1 batch yang diberikan kepada model
Epochs	Jumlah iterasi proses training model ANN
Optimizer	Algoritma atau formula yang digunakan untuk menghitung perubahan weight dan bias dalam proses training model ANN
Regularizer	Teknik atau formula yang digunakan untuk mengubah nilai loss pada proses training untuk mengurangi resiko overfitting atau underfitting

Tabel 2.3 Rangkuman Perbedaan *Parameter* dan *Hyperparameter*

Parameter	Hyperparameter
Nilainya berubah selama proses training tergantung pada data yang digunakan	Nilainya ditentukan sebelum proses training untuk model dilakukan
Berada di dalam model ML	Berada di luar model, merupakan variabel konfigurasi
Nilainya disimpan dan digunakan oleh model yang sudah dilatih	Nilainya tidak berada di dalam model, sehingga tidak digunakan lagi sesudah model dilatih
Nilainya tergantung pada data yang digunakan	Nilainya tidak dipengaruhi langsung oleh data yang digunakan

2.3. Apa Itu *Loss*?

Sebelumnya, kita sudah membahas bahwa *loss* merepresentasikan perbedaan hasil antara *label* dengan *prediction* yang dibuat model ML. Secara definitif, *loss* sebenarnya adalah nilai yang menunjukkan seberapa buruk hasil prediksi dari sebuah model. Secara umum, penelitian dengan ML menggunakan pedoman bahwa nilai *loss* yang besar menandakan ketepatan yang semakin buruk dari model, sehingga *training* dilakukan untuk mengurangi *loss* tersebut. Metode ini disebut juga sebagai *empirical risk minimization*. *Loss* dihitung menggunakan rumus matematika yang berbeda tergantung *task* ML yang dikerjakan. Rumus ini disebut sebagai *loss function*. Dalam subbab ini, kita akan membahas beberapa jenis *loss function* untuk beberapa jenis *supervised learning task*.

Salah satu jenis *task* yang paling umum ditemukan dalam bidang ML adalah *regression*. Kita sudah membahas bahwa *regression* adalah *task* yang menghasilkan nilai kontinu. Untuk kasus seperti ini, beberapa jenis *loss function* yang umum digunakan adalah *Mean Absolute Error* (MAE), *Mean Squared Error* (MSE), dan *Root Mean Squared Error* (RMSE). Dalam anotasi matematis, biasanya *input* ditandai sebagai x , *label* ditandai sebagai y sedangkan hasil prediksi model ditandai sebagai $\hat{y}$ (dibaca y -hat). Semua *loss function*

ini akan membandingkan rata-rata (*mean*) dari selisih nilai y dan $\hat{y}$ dengan untuk setiap sampel dalam dataset dengan jumlah n data menggunakan rumus sebagai berikut.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Seperti yang dapat dilihat dari rumus di atas, MAE menggunakan nilai mutlak dari selisih y dan $\hat{y}$ sedangkan MSE menggunakan nilai kuadratnya. Nilai ini dihitung untuk setiap data, kemudian dirata-ratakan. Nilai MAE dan MSE ini kemudian digunakan sebagai tolak ukur performa *empirical risk minimization* pada model ML untuk regresi. Pada umumnya, MAE dilaporkan bersama dengan nilai MSE atau RMSE dalam evaluasi model ML. Hal ini dikarenakan MSE dan RMSE cenderung lebih sensitif terhadap selisih y dan $\hat{y}$ yang lebih besar. Dengan kata lain, jika kita menguji model regresi dengan nilai MAE yang kecil dengan MSE atau RMSE yang jauh lebih besar, kemungkinan terdapat beberapa hasil prediksi model yang berbeda jauh dari *label* walau mungkin jumlahnya tidak terlalu banyak.

Untuk *task* klasifikasi, *loss function* yang umum digunakan adalah *Cross-Entropy* (CE). CE digunakan karena dalam kasus klasifikasi, y dan $\hat{y}$ bukan berupa nilai kontinu seperti dalam *task* regresi. Karena tujuannya adalah mengklasifikasikan data ke dalam *class* yang sesuai, y dan $\hat{y}$ berupa nilai probabilitas data *input* termasuk ke dalam masing-masing kelas yang bila dijumlahkan nilainya adalah 1. Dalam hal ini, $\hat{y}$ dapat kita simbolkan sebagai probabilitas p . Misalnya kita menggunakan model CNN untuk mengklasifikasikan gambar kucing dan anjing. Kita bisa menganggap kucing sebagai *class* 1 dan anjing sebagai *class* 2, sehingga nilai y untuk kucing adalah $[1,0]$ sedangkan untuk anjing adalah $[0,1]$.

Ketika sebuah CNN melakukan klasifikasi, nilai p bisa berupa $[0.8, 0.2]$, yang berarti CNN tersebut memperkirakan *input* memiliki 80% kemungkinan termasuk sebagai kucing. Untuk menghitung *loss* dari nilai probabilitas ini, kita dapat menggunakan persamaan di bawah ini.

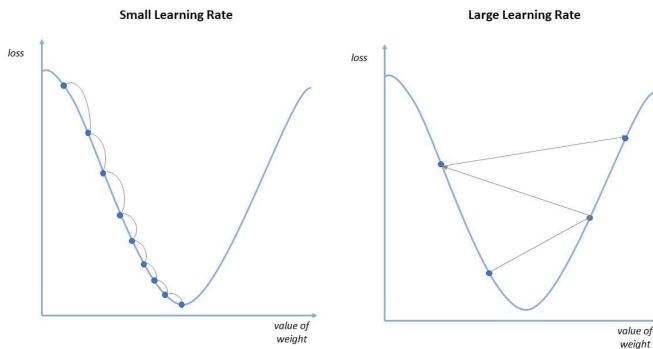
$$CE = - \sum_{i=1}^c y_i \times \log(p_i)$$

di mana c adalah jumlah *class*. Dalam contoh anjing dan kucing tadi, kita dapat menghitung nilai CE seperti ini.

$$CE = -(1 \times \log(0.8) + 0 \times \log(0.2))$$

Sama seperti MAE dan MSE, nilai CE akan dirata-ratakan untuk memperkirakan performa model ML untuk klasifikasi. Rumus yang sama juga digunakan dalam kasus di mana nilai $c > 2$. CE adalah *loss* yang sering digunakan untuk klasifikasi, namun ada banyak *loss function* lain yang dapat digunakan, seperti Kullback-Leibler *divergence*.

2.4. Gradient Descent



Gambar 2.5 Ilustrasi *Gradient Descent* (IBM Cloud Education, 2020)

Dalam ML, *gradient descent* adalah algoritma optimisasi yang digunakan untuk meminimalkan nilai *loss*. Dengan kata lain, ini

adalah metode yang digunakan untuk memperbaharui *knowledge* yang terdapat pada model, termasuk ANN. Rumus dasarnya adalah

$$w_{t+1} = w_t - \gamma \nabla F(x_t)$$

di mana w_t adalah nilai representasi *knowledge* sebelum diperbaharui, w_{t+1} adalah nilainya setelah diperbaharui. γ adalah *hyperparameter* yang disebut *learning rate* sedangkan $\nabla F(x_t)$ adalah apa yang kita sebut sebagai *gradient*. Dalam kasus ANN, w adalah *parameter* seperti *weight* dan *bias*. Hasil perkalian γ dengan *gradient* akan menentukan seberapa besar nilai w harus dirubah, *Gradient* sendiri dihitung menggunakan perhitungan derivatif dari nilai *loss*.

Semakin besar *loss*, nilai *gradient* juga bisa semakin besar. Untuk mengontrolnya, kita umumnya menggunakan *learning rate* yang kecil agar perubahan *knowledge* tidak terlalu besar. Singkatnya, *learning rate* digunakan untuk mengontrol nilai *gradient* agar tidak terlalu kecil (*vanishing gradient*) ataupun terlalu besar (*exploding gradient*). Dalam bidang ML, konsep *gradient descent* ini bisa digambarkan seperti pada **Gambar 2.5**. Nilai *parameter* seperti *weight* yang kurang sesuai bisa menyebabkan nilai *loss* menjadi besar. Karena itu, *gradient descent* dilakukan untuk menentukan nilai *weight* yang optimal untuk menghasilkan *loss* sekecil mungkin.

2.5. Metrik Evaluasi Untuk Model ML

Selain *loss*, biasanya terdapat faktor-faktor lain yang menjadi tolak ukur kualitas model ML. Faktor-faktor yang disebut metrik evaluasi ini memungkinkan kita untuk melakukan analisis secara lebih dalam terhadap model ML. Untuk *task* klasifikasi, contoh metrik yang mungkin paling familiar di telinga kita adalah *accuracy*. Secara matematis, *accuracy* dapat dirumuskan sebagai berikut.

$$accuracy = \frac{TP + TN}{n}$$

di mana TP adalah *True Positives*, yaitu jumlah data yang termasuk ke dalam suatu *class* dan diprediksikan oleh model ke *class* tersebut. Misalnya, jika sebuah gambar kucing diklasifikasikan oleh model ML

sebagai kucing, maka data ini bisa disebut sebuah TP. Sebaliknya, TN atau *True Negatives* adalah jumlah data yang tidak termasuk ke dalam suatu *class* dan diklasifikasikan dengan tepat ke *class* lain. Untuk kasus klasifikasi, metrik-metrik lain yang dapat digunakan meliputi *precision*, *recall*, *F1 score*, serta AUROC (*Area Under Receiver-Operability Curve*). Untuk kasus regresi, contoh metrik yang dapat digunakan adalah *coefficient-of-determination* yang umum ditandai sebagai R^2 (dibaca *R-squared*). Dapat dikatakan bahwa *loss* tidak menjadi satu-satunya metrik penilai kualitas model ML.

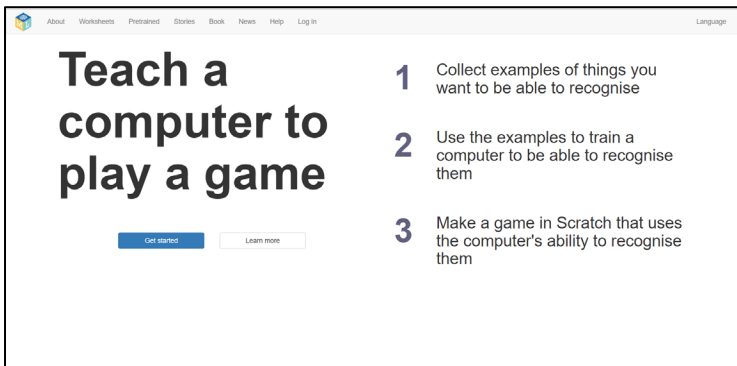
Saatnya Berlatih!

Machine Learning for Kids: Membuat Model ML Tanpa Coding Untuk Pemula

Seperti GTM, *Machine Learning for Kids* adalah sebuah situs *web* yang dapat digunakan untuk membuat model ML tanpa *coding*. Yang membedakannya adalah GTM lebih banyak berfokus ke model DL. Pada latihan kali ini, kita akan mencoba menggunakan *Machine Learning for Kids* untuk membuat sebuah model ML dengan *supervised learning*.

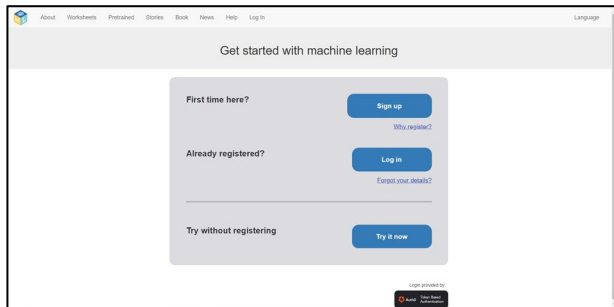
Dalam latihan ini kita akan membuat model ML yang dapat bereaksi terhadap apa yang kita input ke komputer. Kita akan dapat mengontrol perangkat (yang berupa lampu dan kipas angin) dengan menuliskan apa saja sesuai dengan keinginan kita. Kita akan mengajarkan komputer untuk mengenali perintah yang berbeda untuk setiap perangkat dengan memberikan contoh kata/ kalimat masing-masing.

Langkah 1. Buka link <https://machinelearningforkids.co.uk/> pada *browser* anda. Anda akan dibawa ke halaman utama situs.



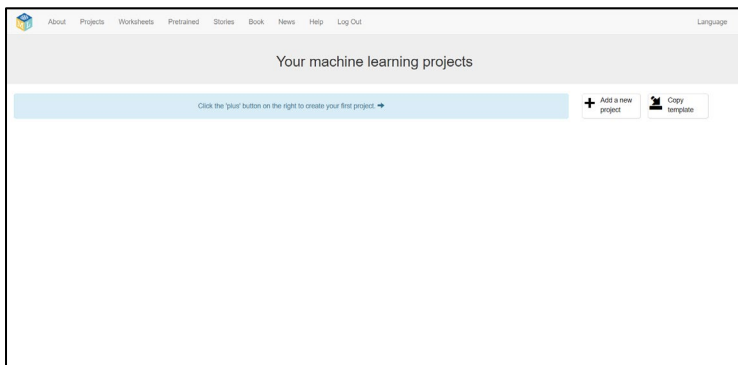
Gambar 2.6 *Screenshot* Tampilan Situs

Langkah 2. Tekan tombol “*Get Started*”. Anda akan diminta untuk melakukan *login*. Namun, kita juga bisa menggunakan situs ini tanpa *login* dengan menekan tombol “*Try it now*” pada bagian yang bertuliskan “*Try without registering*”.



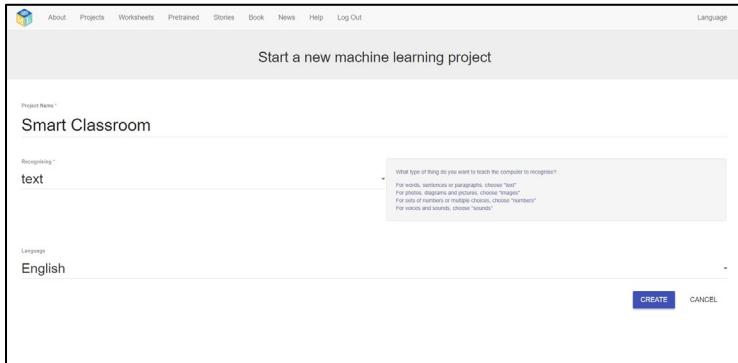
Gambar 2.7 *Screenshot Pilihan Login*

Langkah 3. Tekan tombol “*Add a new project*”. Anda akan diminta untuk memberikan nama pada *project* anda.



Gambar 2.8 *Screenshot Dashboard untuk Memulai Project*

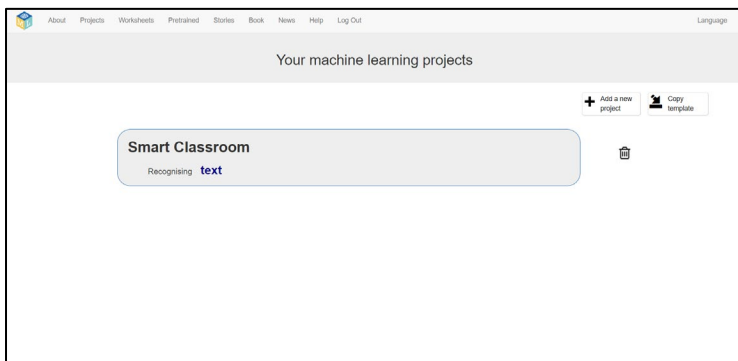
Langkah 4. Mari kita berikan nama “*Smart Classroom*”. Kemudian kita dapat memilih jenis data yang akan menjadi *input* untuk model ML kita. Untuk latihan ini, kita akan memilih “*text* dengan bahasa inggris. Kemudian, tekan tombol “*Create*”.



The screenshot shows a web interface for creating a new machine learning project. At the top, there is a navigation bar with links: About, Projects, Worksheets, Pretrained, Stories, Book, News, Help, and Log Out. A 'Language' dropdown is on the right. The main heading is 'Start a new machine learning project'. Below this, there are three input fields: 'Project Name' with the value 'Smart Classroom', 'Recognizing' with the value 'text', and 'Language' with the value 'English'. A tooltip for the 'Recognizing' field lists options: 'For words, sentences or paragraphs, choose "text"', 'For photos, images and pictures, choose "images"', 'For sets of numbers or multiple classes, choose "numbers"', and 'For names and sounds, choose "sounds"'. At the bottom right, there are 'CREATE' and 'CANCEL' buttons.

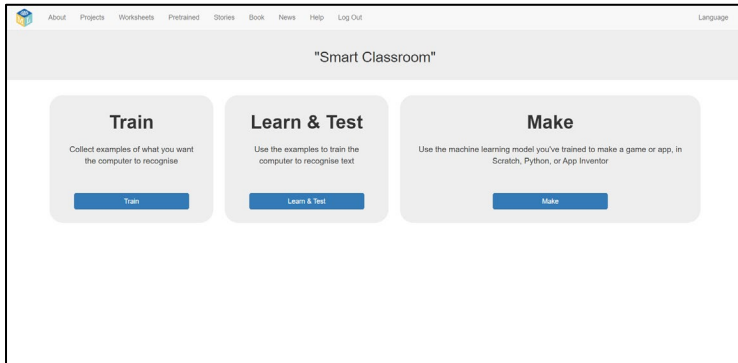
Gambar 2.9 *Screenshot Dashboard Pengaturan Project*

Langkah 5. Sekarang, kita dapat melihat *project* yang baru kita buat pada *dashboard* situs web. Tekan *project* tersebut.



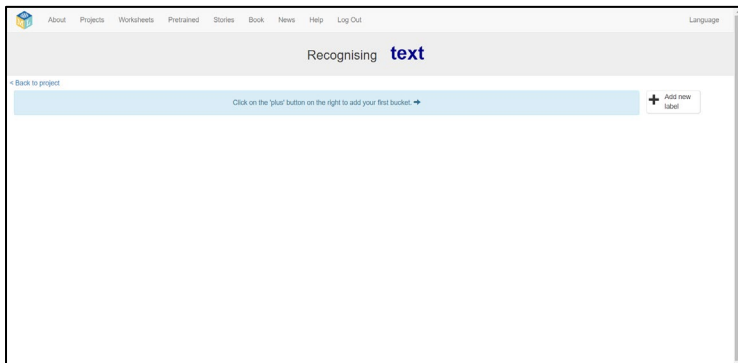
Gambar 2.10 *Screenshot Tampilan Nama Project*

Langkah 6. Kita akan dibawa ke halaman dengan tiga pilihan menu, yaitu “*Train*”, “*Learn & Test*”, dan “*Make*”. Pertama, kita perlu menyusun dataset yang akan kita gunakan untuk melatih model ML kita terlebih dahulu. Maka, tekanlah tombol “*Train*”.



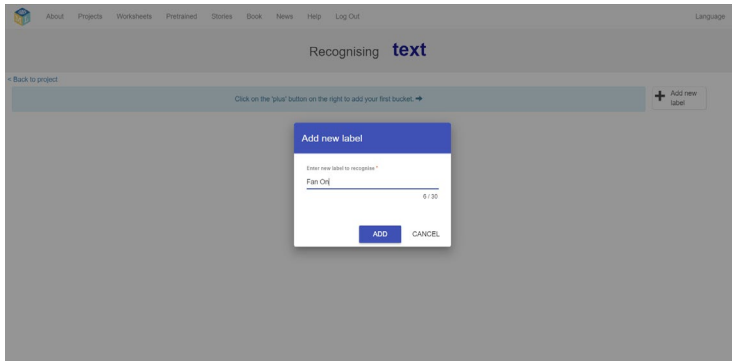
Gambar 2.11 *Screenshot Dashboard Menu pada Project*

Langkah 7. Pertama, mari kita definisikan terlebih dahulu *label* yang akan digunakan untuk data kita. Tekan tombol “*Add a new label*”.



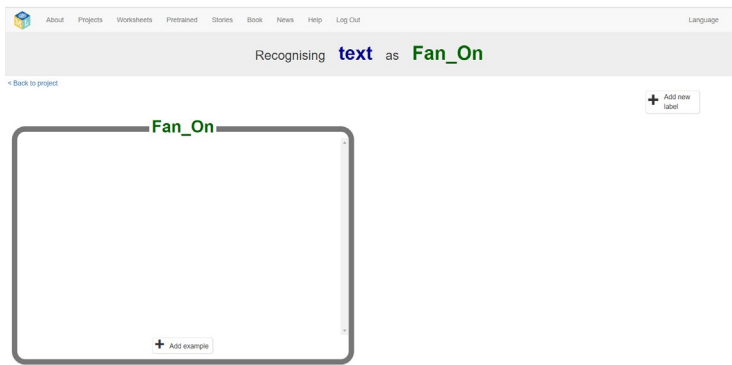
Gambar 2.12 *Dashboard Daftar Label*

Langkah 8. Sekarang, kita dapat menambahkan *label* untuk model ML kita. Model ini akan kita gunakan untuk membuat *Intelligent System* untuk *Smart Classroom* yang dapat mengontrol penggunaan kipas dan lampu. Mari kita tambahkan *label* “Fan On”.



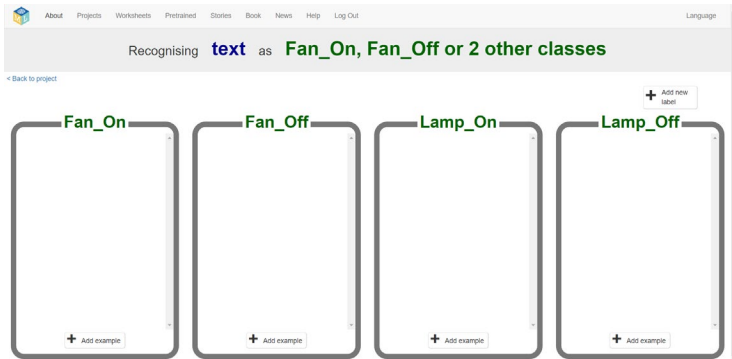
Gambar 2.13 Proses Menambah *Label* Baru

Langkah 9. Sebuah *bucket* akan ditampilkan dengan *label* yang kita tambahkan. Lakukan hal yang sama seperti pada **Langkah 8** untuk menambahkan *label* “Fan Off”, “Lamp On”, dan “Lamp Off”.



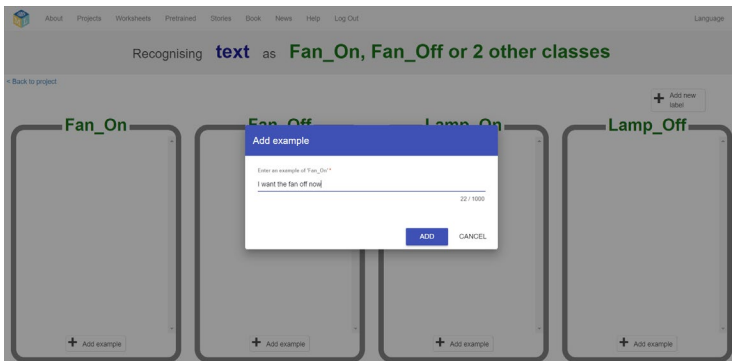
Gambar 2.14 *Screenshot Label* “Fan_On”

Langkah 10. Kita dapat melihat sekarang terdapat 4 *label* sebagai *output* model ML kita nanti. Tekan tombol “Add example” pada bucket “Fan On” untuk menambahkan contoh data yang akan diklasifikasikan ke “Fan On”.



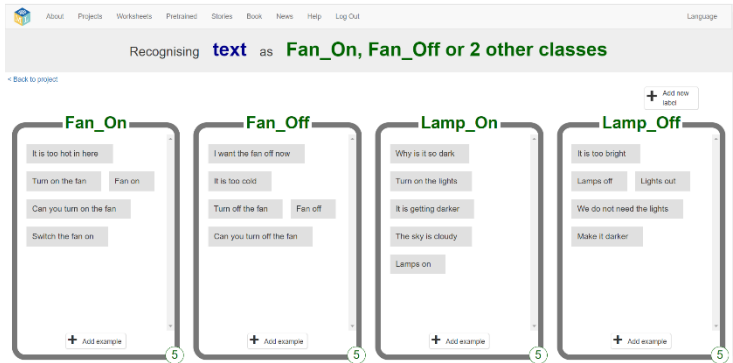
Gambar 2.15 *Screenshot* Tampilan Semua Label yang Digunakan

Langkah 11. Sekarang, kita tambahkan contoh *text* untuk “Fan On”. Misalnya “I want the fan on now”. Lalu tekan tombol “Add”.



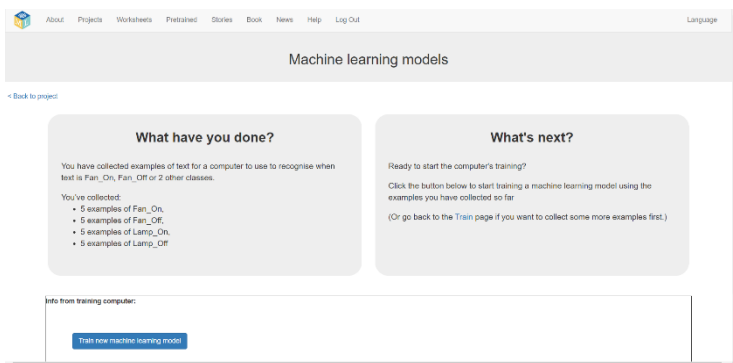
Gambar 2.16 Tampilan Proses Menambah Contoh *Text*

Langkah 12. Lakukan hal yang sama untuk setiap *label* lainnya. Masukkan contoh-contoh kalimat yang dapat digunakan untuk setiap *label* sampai terdapat minimal 5 contoh pada masing-masing *class*.



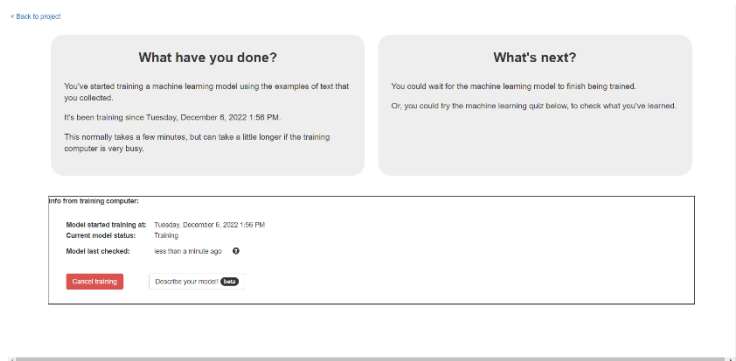
Gambar 2.17 Contoh *Text* Pada Masing-masing *Class*

Langkah 13. Tekan “*Back to project*” untuk kembali ke halaman utama *project* kita. Lalu tekan “*Learn & Test*”. Kita akan dibawa ke halaman untuk melatih model ML kita.



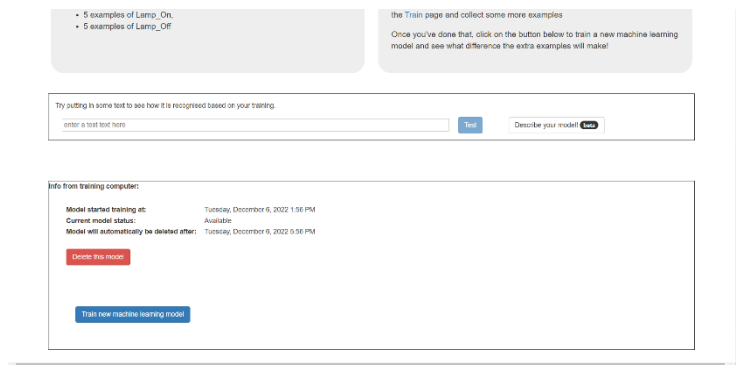
Gambar 2.18 Tampilan *Dashboard Learn & Test*

Langkah 14. Tekan tombol “*Train new machine learning model*”, lalu tunggu beberapa saat.



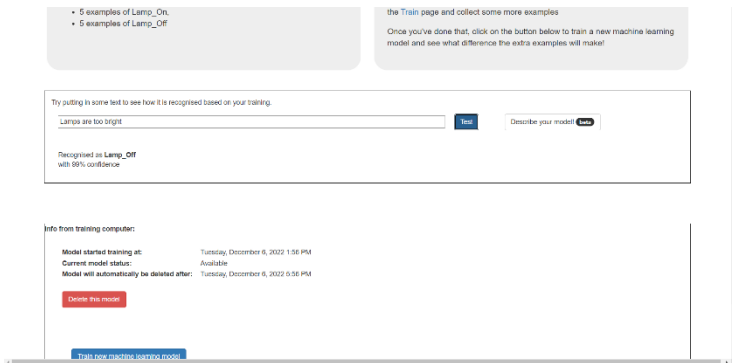
Gambar 2.19 Tampilan Proses *Training*

Langkah 15. Sesudah proses *training* selesai, akan keluar sebuah *text box* karena kita membuat model ML untuk data teks. Sekarang, kita dapat menguji model kita. Masukkan sebuah kalimat ke dalam *text box*, lalu tekan “*test*”. Misalnya kita gunakan kalimat “*Lamps are too bright*”, kalimat yang tidak termasuk di dalam contoh yang kita masukan sebelumnya.



Gambar 2.20 Tampilan Proses *Testing*

Langkah 16. Kita dapat melihat *output* dari model kita dan mengujinya dengan kalimat-kalimat lain. Jika *prediction* yang dihasilkan sudah cukup akurat dan memuaskan, tekan tombol “*Back to project*”.



Gambar 2.21 Tampilan Hasil *Testing*

Wow, kita sudah berhasil membuat model ML yang dapat memproses data teks. Dalam latihan ini, kita sudah menggunakan *supervised learning*. Pada bab berikutnya, kita akan menggunakan data selain gambar dan teks untuk mempelajari lebih dalam pengaruh kualitas data terhadap model ML.